GrabImage_RuntimeCuda Demo 介绍

1. 功能概述

GrabImage_RuntimeCuda 是一个使用 CUDA 运行时 API 从相机获取图像并将其传输到 GPU 内存的示例程序。该项目展示了如何结合工业相机 SDK 与 CUDA 技术进行高效的图像采集和处理。

2. 技术原理

2.1 CUDA 内存管理

- **主机内存分配**：使用 `cudaHostAlloc` 分配主机内存，用于接收相机图像
- **设备内存分配**：使用 `cudaMalloc` 在 GPU 上分配内存，用于存储图像数据
- **内存传输**：使用 `cudaMemcpy` 将图像数据从主机内存传输到设备内存
- **内存释放**：使用 `cudaFree` 和 `cudaFreeHost` 释放设备和主机内存

2.2 相机操作流程

1. 初始化 SDK：`MV_CC_Initialize()`
2. 枚举设备：`MV_CC_EnumDevices()`
3. 创建句柄：`MV_CC_CreateHandle()`
4. 打开设备：`MV_CC_OpenDevice()`
5. 配置参数：`MV_CC_SetIntValueEx()`，`MV_CC_SetEnumValue()` 等
6. 注册缓冲区：`MV_CC_RegisterBuffer()`
7. 开始采集：`MV_CC_StartGrabbing()`
8. 获取图像：`MV_CC_GetImageBuffer()`
9. 释放图像：`MV_CC_FreeImageBuffer()`
10. 停止采集：`MV_CC_StopGrabbing()`
11. 关闭设备：`MV_CC_CloseDevice()`
12. 销毁句柄：`MV_CC_DestroyHandle()`
13. 反初始化 SDK：`MV_CC_Finalize()`

2.3 多线程设计

- 主线程：负责初始化、配置和控制流程
- 工作线程：负责持续获取图像并传输到 GPU

3. 核心流程

- 首先检测系统中可用的 CUDA 设备数量，并选择第一个 CUDA 设备：
- 通过工业相机 SDK 枚举所有连接的相机设备，并允许用户选择要使用的相机：
- 分配 CUDA 主机内存和设备内存
- 创建工作线程，实时获取相机图像并传输到 GPU
- 显示图像采集信息

4. 运行环境要求

- Windows 操作系统
- CUDA 兼容的 NVIDIA GPU
- 工业相机 SDK
- Visual Studio 2017 或更高版本
- CUDA Toolkit

5. 编译与运行

1. 使用 Visual Studio 打开 `GrabImage_RuntimeCuda.sln` 解决方案
2. 编译项目
3. 运行可执行文件
4. 选择要使用的相机设备
5. 程序开始采集图像并将其传输到 GPU
6. 按任意键停止采集
7. 按任意键退出程序